CENTRO UNIVERSITÁRIO SENAC
SANTO AMARO

Alex Shun Yamazaki
Helder Gomes Doutel
Rodrigo da Silva Melo

Classificação de equipamentos utilizando internet das coisas e aprendizado de máquina

São Paulo - Brasil
2019

Alex Shun Yamazaki
Helder Gomes Doutel
Rodrigo da Silva Melo

Classificação de equipamentos utilizando internet das coisas e aprendizado de máquina

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação sob orientação do Prof. Ms. Fábio de Toledo Pereira.

São Paulo - Brasil
2019

Resumo

Neste trabalho foi exercido o uso de Aprendizado de Máquina com dados recebidos via sensores para resolução de um problema de Internet das coisas sobre automação de ambientes com equipamentos não conectados. Com o aumento global na adoção de equipamentos inteligentes/conectados e do crescente investimento em Internet das coisas, uma questão que se torna relevante é a presença de equipamentos não Internet das coisas como uma espécie de legado. Para ter um ambiente conectado à rede atualmente, seria necessário adquirir uma série de produtos que, em comparação com a versão não conectada, são menos acessíveis financeiramente e coletam uma série de informações privadas sobre o ambiente em que está posicionado. Tendo como norte o projeto *Supersensor*, foi produzida uma placa com sensores (som, acelerômetro, luz, interferência eletromagnética), que captura os dados do ambiente e os envia para um servidor responsável por classificar os dados coletados utilizando Aprendizado de Máquina para definir o estado de atividade. A escolha das tecnologias (tanto sensores quanto protocolos e algoritmos) selecionadas neste trabalho, foi feito com base no estudo da solução e através de análise exploratória e prática, não sendo necessariamente iguais às do Supersensor. Com o resultado da classificação, foi possível automatizar o ambiente mesmo onde os equipamentos eram “inteligentes” e monitorar em tempo real o ambiente em que a placa estava conectada à rede elétrica.

Palavras-chave: 1. Aprendizado de máquina. 2. Automação. 3. Internet das Coisas.

Abstract

The purpose of this term paper is to use Machine Learning and data obtained via sensoring to solve IoT problems related to automation on environments that don't have smart equipment. With the global grow in the use of smart/connected equipment and investment being made in IoT devices, a question that pertains is the presence of equipments that were not developed with IoT in mind. Nowadays, in order to create a network connected environment, it would be necessary to acquire a whole new set of equipments that, in contrast with the non-connected versions, are financially less accessible and usually collect private data from the monitored environment. Having the *Supersensor* project as a basis, a board with some sensors (sound, light, electromagnetic interference), that is able to capture data from the environment and send this data over the internet to a server that will be responsible for classifying this data using Machine Learning will be developed. The chosen technologies (sensors, protocols and algorithms) of this project will be selected based on a exploratory analysis, do not being necessarily the same set of tools used on the *Supersensor*. With the classification results in hands, it was possible to automate an environment with non-smart equipment and monitor it in real-time.

Keywords: 1. Machine Learning. 2. Automation. 3. Internet of Things.

Sumário

Lista de ilustrações 7

1 Introdução 11

1.1 Objetivos 12

2 Trabalhos relacionados 13

2.1 Synthetic sensors: towards general-purpose sensing 13

2.2 ElectriSense: single-point sensing using EMI for electrical event detection and classification in the home 15

2.3 Design of a home automation system using arduino 16

3 Revisão bibliográfica 18

3.1 IoT 18

3.1.1 Sistemas embarcados 19

3.1.2 Arquitetura ARM 20

3.1.3 O protocolo MQTT 21

3.1.3.1 Modelo de comunicação Publish/Subscribe (Publicador e Assinante) 21

3.1.3.2 Vantagens e desvantagens do MQTT 22

3.1.4 Sensores 23

3.1.4.1 Acelerômetro MPU6050 23

3.1.4.2 Microfone 24

3.1.4.3 Sensor EMI 25

3.1.4.4 Sensor de luz 25

3.2 Aprendizado de máquina 25

3.2.1 Introdução ao aprendizado de máquina 26

3.2.2 Tipos de aprendizado 27

3.2.2.1 Aprendizado supervisionado 27

3.2.2.2 Aprendizado não supervisionado 28

3.2.3 Classificação versus regressão 28

3.2.4 Tratamento de dados **29**

3.2.4.1 Redução dos dados **30**

3.2.4.2 Ausência e inconsistência de valores **30**

3.2.4.3 Normalização **30**

3.2.4.4 Filtro de Wiener **31**

3.2.5 Escolha do algoritmo de aprendizagem **31**

3.2.6.1 Gradiente descendente **34**

3.2.6.2 Redes neurais **34**

3.2.6.3 Função de ativação **35**

3.2.6.4 Multilayer perceptron **36**

3.2.7 Multi-label classification **37**

4 Desenvolvimento **37**

4.1 Elaboração do circuito e validação dos dados dos sensores (IOT) **40**

4.2 Configuração do broker MQTT **42**

4.3 Recepção dos dados do lado do servidor e treinamento (Middleware) **43**

4.4 Interfaces treinamento e classificação **46**

4.5 Cenários de Teste **47**

5. Conclusão **51**

6 Referências **53**

Lista de ilustrações

- Figura 1 - Placa Supersendor e seus sensores (LAPUT; ZHANG; HARRISON, 2017)
- Figura 2 - Lista de sensores sintéticos do Supersensor e seu peso para a classificação (LAPUT; ZHANG; HARRISON, 2017)
- Figura 3 - Diagrama de blocos dos principais componentes da solução (GUPTA; REYNOLDS; PATEL, 2010)
- Figura 4 - Arquitetura do sistema de casa inteligente (DAVID; CHIMA; UGOCHUKWU; OBINNA, 2015)
- Figura 5 - Arquitetura do hardware da solução (DAVID; CHIMA; UGOCHUKWU; OBINNA, 2015)
- Figura 6 - Exemplo de sistemas embarcados e seus mercados (STALLINGS, 2010)
- Figura 7 - Representação de onda sonora (EARGLE, 2004)
- Figura 8 - Representação de problemas de classificação e de regressão (EDDEL, 2015)
- Figura 9 - Plotagem dados sinal observado, original e estimado (SPORS, 2015)
- Figura 10 - Exemplo de rede neural (MEHRA, 2017)
- Figura 11 - Arquitetura utilizada na solução (autoria própria, 2019)
- Figura 12 - Circuito confeccionado preliminarmente (autoria própria, 2019)
- Figura 13 - Placa de circuitos física (autoria própria, 2019)
- Figura 14 - Acompanhamento dos dados (autoria própria, 2019)
- Figura 15 - Base de dados sem tratamento de número de ocorrências (autoria própria, 2019)
- Figura 16 - Tratamento do som em ambiente doméstico, Azul: sem tratamento, laranja: tratado (autoria própria, 2019)
- Figura 17 - Tratamento do som da CNC, Azul: sem tratamento, laranja: tratado (autoria própria, 2019)

- Figura 18 - Chatbot Telegram para treinamento (autoria própria, 2019)
- Figura 19 - Portal Web no computador (autoria própria, 2019)
- Figura 20 - Circuito finalizado (autoria própria, 2019)
- Figura 21 - Plotagem dos dados oriundos da cozinha (autoria própria, 2019)
- Figura 22 - Matriz de confusão do teste com as CNCs
- Figura 23 - Plotagem dos dados oriundos da oficina (autoria própria, 2019)
- Figura 24 - Matriz de confusão da cozinha com pouca precisão (autoria própria, 2019)
- Figura 25 - Matriz de confusão da cozinha sem filtro de água (autoria própria, 2019)

Glossário

Die: pequeno bloco de material semicondutor, no qual é fabricado o circuito integrado. O Die é o núcleo (não confundir com os cores) principal do processador, onde serão realizados os cálculos.

KNN: K-nearest-neighbor(s) é um algoritmo de aprendizado de máquina.

SVM: Máquina de vetores de suporte, é utilizado para classificação e análise de regressão em aprendizado de máquina.

Timestamp: literalmente, marca temporal, se trata de uma cadeia de caracteres que denotam a hora e ou data que um certo evento ocorreu.

Machine-To-Machine (M2M): trata-se uma tecnologia que permite que sistemas com ou sem fio se comuniquem com outros dispositivos que possuam a mesma habilidade.

Threshold: trata-se de um limite que é estabelecido para algum determinado fenômeno.

TLS/SSL: são protocolos de rede que fornecem acesso a recursos de criptografia e segurança.

TCP: é um protocolo de rede vastamente utilizado nos dias de hoje, é comumente conhecido por TCP/IP.

Early Adopters: grupo de pessoas que querem adquirir um produto ou serviço novo, mesmo que tenha alguns bugs e problemas.

Store-and-Forward: trata-se uma de técnica de telecomunicações em que a informação é enviada a uma estação intermediária onde ela é mantida para ser posteriormente enviada para um dispositivo de destino ou para outra estação intermediária

XML: do inglês *"Extensible Markup Language"* é uma linguagem de marcação utilizada para descrever diversos tipos de dados

QoS: o termo Qualidade de Serviço (QoS, em inglês, Quality of Service), em redes de comutação de pacotes refere-se à garantia de largura de banda ou, como em muitos casos, é utilizada informalmente para referir a probabilidade de um pacote circular entre dois pontos de rede.

1 Introdução

Com o avanço da tecnologia, um assunto que se tornou relevante mundialmente é a automação residencial, já que está se tornando financeiramente viável. No relatório do Gartner com predições sobre casas conectadas a partir de 2017 Marian, Hetu, Joliet e Burt previram que:

> Em 2020, o mercado global de casas inteligentes deverá crescer para pouco menos de US $ 60 bilhões, e uma casa familiar em um mercado maduro e rico poderá conter várias centenas de objetos “inteligentes” (2016).

Para ter um ambiente conectado à rede atualmente, é necessário adquirir uma série de produtos (e se desfazer dos equivalentes não conectados) que, em comparação com a versão não conectada, são menos acessíveis financeiramente. Além disso, os dispositivos *IoT* (geladeira, *home assistant*, aquecedor e etc) coletam uma vasta gama de informações do ambiente em que estão e reportam aos seus respectivos fabricantes (HILL; MATTU, 2018), o que pode desagradar os usuários que desejam o conforto trazido pela automação doméstica sem abrir mão da privacidade de seus dados.

Este tipo de problema da área de *IoT* fomenta a realização de diversos trabalhos científicos (como visto nos Trabalhos Relacionados). Dentre eles, é possível ressaltar o *Supersensor* (LAPUT; ZHANG; HARRISON, 2017) que vai ao encontro dos objetivos pretendidos neste trabalho ao promover uma solução para o problema de classificação e monitoramento de equipamentos não conectados.

Este trabalho foi desenvolvido baseado na solução Supersensor, funcionando como uma prova de conceito pois, enquanto o Supersensor utiliza 12 sensores diferentes, este projeto foi feito utilizando os 4 de maior peso na classificação (de acordo com levantamento realizado no projeto Supersensor). O projeto consiste em uma placa com uma série de sensores que é conectada à rede elétrica e funciona como um ponto zero para recepção de dados do ambiente que depois de coletados e tratados são utilizados com Aprendizado de Máquina para reconhecimento e classificação de equipamentos / eventos permitindo a monitoração de um ambiente

sem a necessidade de posicionar sensores em cada equipamento que se deseja classificar. Resolvendo assim o problema de classificação e monitoramento de equipamentos não conectados de forma mais acessível financeiramente, com maior privacidade e de forma mais centralizada.

1.1 Objetivos

Enumerando os objetivos alcançados no decorrer do projeto, primeiramente foi a produção funcional do projeto, passando pela captação dos dados, do tratamento, da modelagem, do treinamento e finalmente da classificação com sucesso, em tempo real de alguns equipamentos. Em seguida o objetivo foi comparar os resultados com os observados no Supersensor para definir a precisão e assertividade além de entender as diferenças.

Com isso, foi possível mostrar como o projeto pode ser utilizado em diversos contextos diferentes através de Aprendizado de Máquina e treinamento do modelo, podendo se adaptar tanto à uma oficina quanto à um ambiente domiciliar.

O projeto não compromete a privacidade dos dados daqueles presentes no ambiente pois não trafega dados “sensíveis” como imagem, vídeo ou trechos de áudio mantendo a privacidade dos usuários mesmo se os dados forem interceptados.

Sendo assim, os objetivos foram:

1. Realizar a prova de conceito conseguindo efetivamente classificar os equipamentos em um ambiente qualquer.
2. Comparar o desempenho do projeto com o do Supersensor e identificar as diferenças.

2 Trabalhos relacionados

Nesta etapa do projeto, será dissertado sobre trabalhos que visam resolver um problema semelhante com o presente trabalho, porém utilizando outras abordagens. Foram analisados três trabalhos com ideias e resultados relevantes para a área de automação residencial e conectividade de equipamentos não *IoT*.

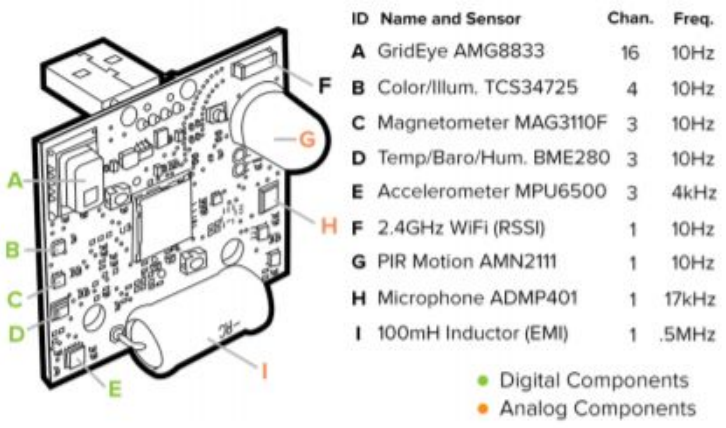
2.1 Synthetic sensors: towards general-purpose sensing

Projeto desenvolvido por Gierad Laput, Yang Zhang e Chris Harrison de uma placa com os seguintes sensores: Acelerômetro, Microfone, EMI, Temperatura, Barômetro, Umidade, Luz, Cor, Magnético, RSSI, IR e GridEye cujo propósito é utilizar aprendizado de máquina para reconhecimento de padrões.

Ao invés de se basear nos dados recebidos de um único sensor acoplado no equipamento que se deseja monitorar, o Supersensor utiliza o conceito de *Sensores Sintéticos* que são sensores virtuais formados pelos dados obtidos dos diversos sensores acoplados.

Na Figura 1 estão apontados todos os sensores utilizados na placa.

Figura 1 - Placa Supersensor e seus sensores



Fonte: Laput; Zhang; Harrison (2017)

O *Supersensor* serviu de base para o nosso trabalho, ele possui uma maior gama de sensores que melhora a precisão da classificação e aumenta a quantidade de equipamentos / situações que podem ser identificadas, como retratado na Figura 2 contendo o peso de cada sensor na classificação:

Figura 2. Lista de sensores sintéticos do Supersensor e sua relevância para a classificação

ID	Sensor	Accel	Mic	EMI	Temp	Baro	Humid	Light	Color	Mag	RSSI	IR	GEye
A	Kettle On	9%	9%	3%	4%	3%	2%	2%	4%	7%	2%	2%	55%
B	Paper Towel Dispensed	61%	36%	-	-	-	-	-	-	-	-	-	-
C	CO Detector Alarming	73%	5%	2%	3%	2%	-	-	2%	4%	1%	1%	6%
D	Dishwasher Running	59%	39%	-	-	-	-	-	-	-	-	-	-
E	Faucet Running	37%	59%	-	-	-	-	-	-	-	-	-	-
F	Fridge Door Closed	77%	5%	1%	2%	1%	-	-	2%	3%	1%	-	5%
G	Soap Dispensed	54%	43%	-	-	-	-	-	-	-	-	-	-
H	Dishwasher Done Chime	74%	6%	1%	2%	2%	-	-	2%	4%	1%	-	5%
I	Door Closed	49%	47%	-	-	-	-	-	-	-	-	-	-
J	Projector Running	36%	12%	2%	4%	3%	2%	4%	19%	9%	2%	2%	5%
K	Person Speaking	20%	23%	2%	3%	2%	1%	5%	28%	10%	2%	1%	3%
L	Room Lights Off	19%	21%	2%	3%	2%	1%	6%	35%	7%	2%	1%	3%
M	Room Lights On	15%	11%	2%	3%	2%	1%	10%	35%	15%	2%	1%	4%
N	Microwave Running	41%	26%	27%	-	-	-	-	-	1%	-	-	-
O	Coffee Maker Brewing	32%	29%	28%	2%	1%	-	-	1%	3%	-	-	2%
P	Microwave Keypad Press	37%	50%	10%	-	-	-	-	-	-	-	-	-
Q	Microwave Door Opened	46%	39%	11%	-	-	-	-	-	-	-	-	-
R	Microwave Door Closed	41%	26%	28%	-	-	-	-	-	1%	-	-	-
S	Toaster Done Beep	27%	56%	11%	-	-	-	-	-	1%	-	-	1%
T	W. Fountain Chiller Active	23%	49%	19%	1%	-	-	-	1%	2%	-	-	2%
U	Smoke Detector Alarming	27%	56%	11%	-	-	-	-	-	1%	-	-	1%
V	Water Fountain Dispensing	42%	46%	10%	-	-	-	-	-	-	-	-	-
W	Microwave Done Chime	58%	15%	16%	2%	1%	-	-	1%	3%	-	-	2%
X	Phone Ringing	60%	21%	2%	3%	2%	-	1%	2%	4%	1%	1%	3%
Y	Knock on Door	56%	37%	-	-	-	-	-	-	2%	-	-	1%
Z	Door Closed	56%	36%	-	-	-	-	-	-	2%	-	-	1%
α	Person Talking	52%	37%	-	1%	-	-	-	1%	2%	-	-	2%
β	Phone Loud Speaker On	49%	40%	-	1%	-	-	-	1%	2%	-	-	2%
γ	3D Printer Running	44%	54%	-	-	-	-	-	-	-	-	-	-
δ	Paper Towel Dispensed	82%	6%	-	2%	1%	-	-	1%	2%	-	-	2%
ε	Hammering	53%	46%	-	-	-	-	-	-	-	-	-	-
ζ	Wood Saw Running	45%	53%	-	-	-	-	-	-	-	-	-	-
η	Dremel Running	79%	4%	1%	2%	2%	-	-	2%	4%	1%	-	3%
θ	Shop Vac Running	47%	52%	-	-	-	-	-	-	-	-	-	-
λ	Dust Gorilla Running	63%	36%	-	-	-	-	-	-	-	-	-	-
μ	Power Drill Running	43%	55%	-	-	-	-	-	-	-	-	-	-
ξ	Sander Running	45%	54%	-	-	-	-	-	-	-	-	-	-
π	Laser Cutter Running	58%	41%	-	-	-	-	-	-	-	-	-	-
-	Null Event	-	-	-	-	-	-	-	-	-	-	-	-

Fonte: Laput; Zhang; Harrison (2017)

Como é possível ver na Figura 2, para cada “evento” a relevância dos dados captados pelos sensores é distinta, podendo ter alta relevância ou até mesmo relevância nenhuma. Para a nossa prova de conceito por exemplo, utilizamos os sensores de maior peso para a classificação (acelerômetro, microfone, e EMI).

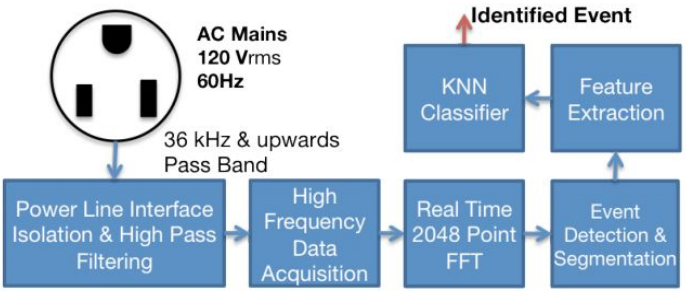
Durante o estudo Laput, Zhang e Harrison constataram que a melhor abordagem para o treinamento do modelo deles é o treinamento supervisionado que requer acompanhamento durante a fase de treino porém entrega uma precisão relativamente maior em comparação com o treinamento não supervisionado. Mesmo assim, um ponto que pode ser trabalhado futuramente é a necessidade de supervisão no treinamento do algoritmo, podendo extrapolar para treinamento não supervisionado e até mesmo utilizando uma base global de dados com o resultado do treinamento realizado em diversos locais (empresas, residências, fábricas) melhorando a abrangência modelo.

2.2 ElectriSense: single-point sensing using EMI for electrical event detection and classification in the home

O projeto *ElectriSense* é outro trabalho relacionado à área de classificação do uso de equipamentos (conectados à rede elétrica) em determinados ambientes utilizando, neste caso, um sensor EMI (interferência eletromagnética) e algoritmos de Aprendizado de Máquina*,* onde dessa forma foi possível realizar a classificação do uso de aparelhos aprendendo os padrões através do gasto de energia identificado de um único ponto na rede elétrica. A demanda de energia somada ao tempo de uso e outros parâmetros é utilizada para reconhecer equipamentos em uso.

Como é possível ver no diagrama da solução na figura 3, partindo da rede elétrica é feito o isolamento e filtragem que possibilitam a aquisição dos dados de alta frequência que são coletados, detectados, segmentados até ter suas características extraídas para que o classificador KNN identifique o evento.

Figura 3. Diagrama de blocos dos principais componentes da solução.



Fonte: Gupta; Reynolds; Patel (2010)

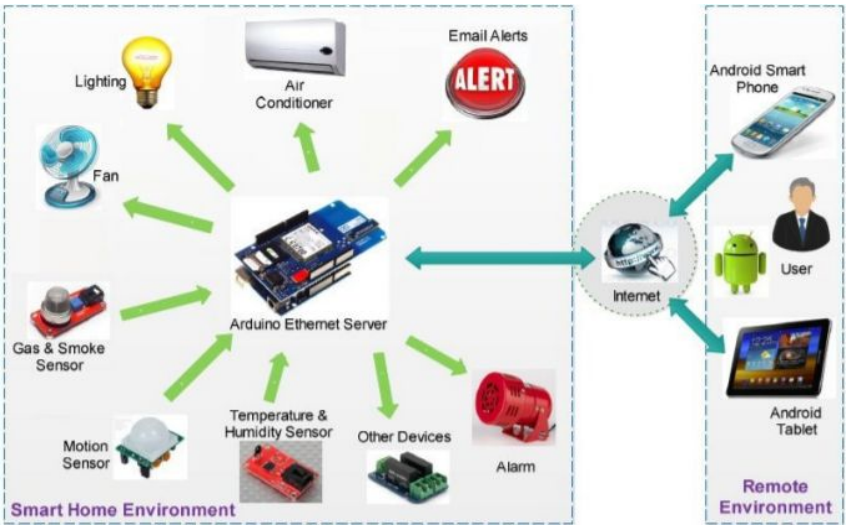
Este projeto conseguiu alcançar uma precisão de 93,82% na detecção / identificação de eletrodomésticos ligados na rede elétrica, uma taxa interessante mesmo que só possa ser utilizado em equipamentos ligados à rede elétrica. As variações recebidas via EMI são mapeadas para instâncias com poucas características / atributos, o que condiz com o uso do algoritmo *KNN* utilizado neste trabalho.

2.3 Design of a home automation system using arduino

Este projeto foi desenvolvido por David, Chima, Ugochukwu e Obinna e além de monitoramento, foca no controle remoto dos equipamentos através de interruptores integrados aos equipamentos conforme necessário para formar um sistema de controle residencial e de monitoração de ambiente. A solução proposta tem as opções de controlar os equipamentos via bluetooth ou app android subindo na própria placa arduino um micro servidor web com PHP para integração e comunicação com os aparelhos.

Na imagem a seguir, é possível enxergar que o controle das luzes, ventiladores, ar condicionado, alarme residencial e etc ficaria mais centralizado e prático:

Figura 4 - Arquitetura do sistema de casa inteligente

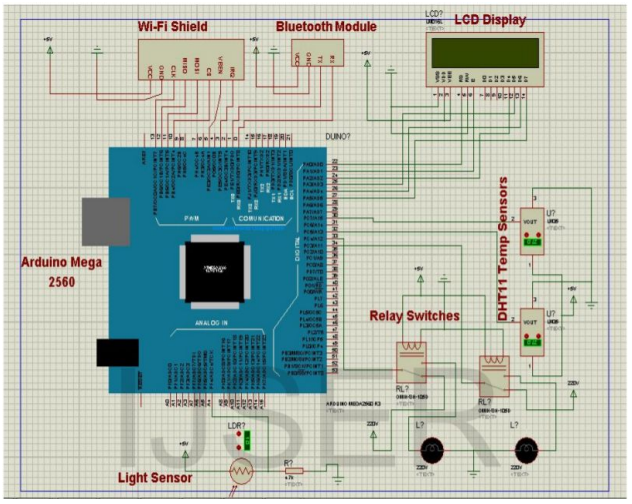


Fonte: David; Chima; Ugochukwu (2015, p797)

Como resultado, foi possível automatizar e monitorar diversos equipamentos de forma simples através de dispositivos móveis com conexão à rede sem fio. A arquitetura funcionou e correspondeu ao esperado. Foi proposto que futuramente sejam feitas melhorias como alertas via SMS e afins de acordo com o resultado da monitoração.

Neste trabalho também é possível ver como David, Chima, Ugochukwu e Obinna montaram a placa arduino e integraram à solução completa com os sensores e interruptores, o que foi muito útil no decorrer deste projeto.

Figura 5 - Arquitetura do hardware da solução



Fonte: David; Chima; Ugochukwu (2015, p798)

3 Revisão bibliográfica

Para facilitar a compreensão e desenvolvimento do produto final, as áreas de conhecimento que foram utilizadas no projeto foram divididas em três segmentos que servem de base para alcançar a proposta deste trabalho. Começando pela explicação do conceito de IoT, dos sistemas embarcados, arquitetura *ARM*, além de uma breve abordagem de alguns protocolos e arquiteturas *IoT*, seguido pelos sensores que foram responsáveis pela captação dos dados brutos, e finalizando com Aprendizado de Máquina que é o núcleo da solução, sendo responsável pela classificação dos dados.

3.1 IoT

IoT se refere ao termo *Internet of Things,* em português, Internet da Coisas. IoT é um novo paradigma em que os objetos que fazem parte do dia-a-dia das pessoas se comunicam entre si por meio da Internet (LEE; LEE, 2015). De acordo com In Lee; Kyoochun Lee (2015), teremos cerca de 26 bilhões de dispositivos conectados até 2020.

Com o surgimento desse novo paradigma, surge também um grande volume de dados e novas necessidades de comunicação para integrar a quantidade de dispositivos conectados.

In Lee; Kyoochun Lee (2015) dividem o IoT em cinco grandes tecnologias necessárias para implementação de uma solução, que são:

- Identificação por radiofrequência: Permite a comunicação por tags, capazes de armazenar diversas informações e transmitir automaticamente para os dispositivos próximos.
- Redes de sensores conectados: Consiste em uma rede de sensores autônomos capazes de monitorar o meio e se comunicar com outros dispositivos.

- *Middleware*: Camada de software que conecta os dispositivos sem a necessidade de implementações específicas para cada um dos sensores
- Computação em nuvem: Camada responsável por alocar as aplicações de forma a oferecer um sistema menos suscetível a falhas, escalável e distribuído, devido ao grande volume de dados produzidos no IoT
- *Softwares* para IoT: Responsável pela interação entre homem-máquina e máquina-máquina.

Existem diversas tecnologias que visam atender aos requisitos citados acima e seus desafios. A solução desenvolvida neste trabalho se encaixa principalmente na parte de softwares para IoT pois melhora a interação homem-máquina (classificação em tempo real no portal web) e máquina-máquina (automação da atividade de um equipamento baseado na classificação).

Este tópico apresenta alguns pontos sobre sistemas embarcados e o baixo consumo de energia em conjunto com as arquiteturas e protocolos de comunicação para a transmissão dos dados obtidos e uma breve visão sobre os principais sensores utilizados.

3.1.1 Sistemas embarcados

Sistemas embarcados se referem ao uso de *hardware* e *software* em uma determinada aplicação especifica (produto) e podem ter diversas aplicações diferentes.

Geralmente, os sistemas embarcados servem a uma determinada função no ambiente em que estão sendo aplicados, utilizando diversas formas de receber dados que variam dos meios convencionais (mouse, teclado e etc). Em um sistema embarcado, a entrada dos dados pode ser feita através de sensores, que medem algum aspecto do ambiente e tomam alguma ação, ou fazem parte de um sistema maior (não embarcado) (STALLINGS, 2010).

Figura 6 - Exemplos de sistemas embarcados e seus mercados

Mercado	Dispositivo embutido
Automotivo	Sistema de ignição Controle de motor Sistema de freio
Eletrônico (para consumo)	Televisores digitais e analógicos Caixas set-top (DVD, VCR, cabo) *Personal Digital Assistants* (PDA) Aparelhos de cozinha (refrigeradores, torradeiras, fornos de micro-ondas) Automóveis Brinquedos/jogos Telefones/celulares/pagers Câmeras Sistemas de posicionamento global (GPS, do inglês *global positioning systems*)
Controle industrial	Robótica e sistemas de controle para manufatura Sensores
Médico	Bombas de infusão Máquinas de diálise Dispositivos protéticos Monitores cardíacos
Automação de escritório	Máquinas de fax Fotocopiadoras Impressoras Monitores Escâneres

Fonte: Stallings (2010, p36)

3.1.2 Arquitetura ARM

A arquitetura ARM aparece diversas vezes no contexto de sistemas embarcados (descrito acima) devido suas características conforme descrito por Stallings (2010):

> Os chips ARM são processadores de alta velocidade, conhecidos por seu pequeno tamanho do *die* e baixos requisitos de potência. Eles são bastante utilizados em PDAs e outros dispositivos portáteis, incluindo jogos e telefones, além de uma grande variedade de produtos para o consumo. Chips ARM são os processadores presentes nos populares dispositivos iPod e iPhone da Apple. A ARM provavelmente é a arquitetura de processador embutido mais utilizada e certamente a arquitetura de processador mais utilizada de qualquer tipo no mundo.(2010, p.37).

Os chips *ARM* são projetados para as seguintes aplicações: Sistemas embarcados em tempo real, Plataforma de aplicações e Aplicações seguras (STALLINGS, 2010).

O projeto *ARM* tem como característica combinar o desempenho com baixo consumo de energia, tamanho pequeno e baixo custo.

3.1.3 O protocolo MQTT

O protocolo *MQTT* (do inglês, "*Message Queue Telemetry Transport*") foi criado pela IBM voltado para comunicações *M2M* de forma leve (ou seja, sem sobrecarregar a rede de transporte), através de um protocolo assíncrono baseado no modelo *Publish/Subscribe* (explicado na seção 3.1.3.2).

Esse protocolo se baseia na comunicação via *broker* com tópicos, *publishers* e *subscribers* (abordados também na seção 3.1.3.2). Esse protocolo é voltado para ambientes em que os dispositivos e as redes são limitados, podendo ser usado entre dispositivos que se comunicam através de redes intermitentes e, por ser um protocolo que não sobrecarrega a rede, é recomendado em cenários onde os dados trocados devem possuir o menor tamanho possível (CIRANI; FERRARI; PICONE; VELTRI, 2019). Conforme estudos realizados e indicação do orientador deste trabalho foram mensuradas as vantagens e desvantagens do MQTT e do modelo Publish/Subscribe na sua escolha ao invés das alternativas possíveis (no *paper* do projeto Supersensor não foi especificado qual o protocolo/modelo utilizado).

3.1.3.1 Modelo de comunicação Publish/Subscribe (Publicador e Assinante)

No modelo de comunicação *Publish/Subscribe*, é possível definir dois tipos de entidades: *publishers* e *subscribers*. Os publishers enviam mensagens para um "tópico" no servidor e os *subscribers* podem se inscrever nesses tópicos para receber uma cópia de todas as mensagens enviadas para esses tópicos (ou seja, uma mensagem pode ser recebida por mais de um *subscriber*) (CIRANI; FERRARI; PICONE; VELTRI, 2019).

O intermediário entre um *publisher* e um *subscriber* é chamado de *broker*, que é um servidor responsável por receber mensagens dos *publishers* em determinados tópicos e replicá-las para todos os *subscribers*.

Um exemplo simples do funcionamento do *MQTT* pode ser feito pensando em uma solução que capta dados de umidade e temperatura de um determinado ambiente e envia para um servidor analisar. No qual o dispositivo que capta os dados seria um *publisher* utilizando os tópicos *sensor1/umidade* e *sensor1/temperatura* e o servidor seria um *subscriber* que utilizaria os mesmos tópicos para receber os dados.

3.1.3.2 Vantagens e desvantagens do MQTT

Existem algumas vantagens e desvantagens do modelo de comunicação *Publish/Subscribe* e do protocolo *MQTT*. Cirani, Ferrari, Picone e Veltri (2019) definem as seguinte vantagens:

- **Desacoplamento:** os *publishers* não precisam de nenhuma informação relativa aos *subscribers* (informações como existência ou não do *subscriber* e confirmação de recepção de mensagem);
- **Escalabilidade:** o fato de os *brokers* apenas direcionarem mensagens torna sua replicação para suportar maiores volumes de dados, uma tarefa relativamente simples;
- **Implementação:** vastamente utilizado na maioria das plataformas e linguagens de programação do mercado; e
- **Assincronismo:** o *MQTT* trabalha com comunicação assíncrona devido ao desacoplamento entre *publisher* e *subscriber*, e essa característica permite um fluxo de transmissão de dados estável (pois não existe necessidade de confirmação de recepção de mensagens por nenhuma das partes).

E as seguintes desvantagens:

- **Segurança:** não existe suporte para segurança ponta-a-ponta entre os *publishers* e *subscribers*. Porém as mensagens podem ser criptografadas e

descriptografadas utilizando chaves e caso os dados sejam interceptados o conteúdo em si não carrega nenhum dado pessoal como imagem ou trecho de áudio; e

- **Controle de escalabilidade:** a escalabilidade da infraestrutura do *broker* precisa crescer de forma a suportar o carga de mensagens que chegam e são enviadas pelo *broker*, pois isso pode causar lentidão na troca de mensagens.

3.1.4 Sensores

Os sensores são componentes que transformam fenômenos físicos em pulsos elétricos que podem ser lidos de forma digital ou analógica por diversos tipos de dispositivos para assim serem processados.

Existe uma vasta quantidade de sensores no mercado, que podem ser aplicados para diversas finalidades, desde monitoramento voltado para segurança até utilidades do dia-a-dia (Ex.: detector de umidade de solo, detector de fluxo de água e consumo de energia elétrica).

Para a solução proposta neste trabalho, foram escolhidos os sensores abaixo levando em conta sua utilidade para a captação de dados e relevância conforme a figura 2.

3.1.4.1 Acelerômetro MPU6050

O Acelerômetro é importante pois capta pressão no ambiente de forma tridimensional através de seus eixos.

De acordo com a documentação MPU-6000/MPU-6060 EV Board User Guide, o acelerômetro é um componente que converte movimento mecânico em pulsos elétricos que variam de acordo com a intensidade das forças aplicadas.

O MPU 6050 é um sensor de 6 eixos com baixo consumo de energia que consegue entregar uma alta precisão nos dados apresentados. O MPU 6050 tem como saída 16 bits para o acelerômetro.

Como descrito na documentação do sensor, para manter o baixo consumo de energia, o sensor mantém um fila de 1024 bytes que possibilita o processador do sensor a ler todos os dados de uma vez e em seguida entrar em modo de economia de energia, portanto não precisando processar os sinais o tempo todo.

O MPU 6050 fornece o processamento dos dados no chip, evitando que o *hardware* principal tenha que processar as informações.

3.1.4.2 Microfone

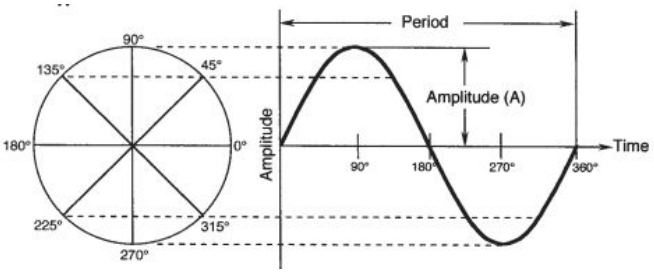
O microfone está presente pois as variações nas ondas sonoras são insumos de grande relevância para a classificação através de sua amplitude e intensidade.

Para entender os fundamentos do microfone é necessário entender o que é o som em sua essência.

O som é transmitido em forma de ondas pelo meio, a amplitude se dá a partir da distância de uma crista até a outra, ou seja, do seu valor máximo até atingir seu próximo pico. O período se refere ao tempo necessário para a onda sair do seu máximo e chegar novamente a um novo ciclo. A frequência se refere a quantidade de ciclos que são realizados em um determinado espaço de tempo. Quando um som é emitido, o meio propaga a onda em forma de vibração (ondas).

Com esses conceitos em mente, é possível entender o conceito básico de como o microfone deve atuar. Quando a onda se propaga, temos um movimento mecânico acontecendo no meio. O microfone é o equipamento capaz de captar o movimento mecânico e traduzir em sinais elétricos mantendo suas características de frequência e curvas, que podem ser transmitidos em outros meios, como rádios, gravações etc (EARGLE, 2004). Na figura 7 está a representação de uma onda sonora com a amplitude da onda em relação ao tempo.

Figura 7 - Representação de onda sonora



Fonte: Eargle(2004, p8)

3.1.4.3 Sensor EMI

Este sensor é útil principalmente na classificação de equipamentos que emitam interferência eletromagnética como aparelhos eletrônicos por exemplo.

EMI (do inglês, *electromagnetic interference*) é o termo utilizado para interferência eletromagnética que se propaga em forma de radiação eletromagnética. Aparelhos que utilizam eletricidade emitem radiação eletromagnética, alguns mais e outros menos, dependendo de como foram projetados e se existe algum tipo de proteção para bloquear esse tipo de emissão (CARR, 2000).

A radiação eletromagnética é uma combinação de eletricidade e magnetismo em forma de ondas, quando esse sinal é propagado no meio, é gerado uma alteração de corrente nos aparelhos próximos, com essa característica é possível receber a radiação em forma de eletricidade e medir o nível de interferência que está sendo gerada (GERTZ; JUSTO, 2012).

3.1.4.4 Sensor de luz

O sensor de luz capta a intensidade de luz aplicada nele então fornece os dados da intensidade da luz no ambiente.

De acordo com o *datasheet* do sensor de luz modelo 3190, o sensor de luz, conhecido com *LDR* (*Light Dependent Resistor*) é um tipo de resistor feito de um semicondutor que quando exposto à luz resiste menos a passagem de corrente. Sendo que a partir desse comportamento, é possível extrair um valor numérico que representa a intensidade da luz do ambiente em que o sensor se encontra.

3.2 Aprendizado de máquina

A proposta deste trabalho também aborda o tema Aprendizado de Máquina, pois é a partir do estudo deste que a classificação de eventos de fins gerais foi realizada.

Os tópicos desta seção buscam introduzir alguns dos conceitos básicos de Aprendizado de Máquina, como também algumas informações adicionais que foram consideradas relevantes ao trabalho que foi desenvolvido.

3.2.1 Introdução ao aprendizado de máquina

Hoje em dia, vivemos em uma sociedade dependente dos computadores para a realização de diversas tarefas repetitivas que possuem diferentes níveis de importância, e que em muitos casos são realizadas com altos padrões de qualidade em comparação aos seres humanos.

Um exemplo simples de uma dessas atividades são os processamentos que devem ser realizados rotineiramente por um sistema bancário, que pelo fato de estar lidando com grandes quantidades monetárias, não deve assumir riscos que possam vir a acontecer devido à erros humano.

Porém ao longo dos anos foi se percebendo que apesar de os computadores serem máquinas extremamente úteis e eficazes para a solução de muitos problemas, ainda existiam outros problemas em que se tornava necessária a supervisão humana parcial ou até mesmo total, e é nesse contexto que o aprendizado de máquina, mais conhecido pelo seu termo em inglês *"Machine Learning"* apareceu para diminuir ou até mesmo eliminar a necessidade de um elemento humano na fórmula de resolução desses problemas.

De acordo com Andriy Burkov (2019) o termo *"Machine Learning"*, faz na verdade, uma analogia com a forma que os animais (incluindo os seres humanos) aprendem a partir do que lhes é ensinado, não necessariamente seu significado literal. O autor também cita um exemplo que se trata de uma situação onde um ser-humano e um algoritmo aprendem a jogar vídeo-game, onde o ser-humano pode se adaptar a diversos fatores como a rotação da televisão e o algoritmo não necessariamente conseguirá se adaptar, já que pode ter sido treinado com algumas premissas, como a fixação da posição da televisão, por exemplo.

Ainda de acordo com Andriy Burkov (2019), *Machine Learning* trata-se de uma subárea da Ciência da Computação que está relacionada em construir

algoritmos que para serem funcionais dependem de uma coleção de exemplos de algum fenômeno, sendo que esses exemplos podem vir da natureza, serem feitos manualmente por humanos ou até mesmo gerados por algum outro algoritmo. O autor ainda cita que o termo também pode ser definido como o processo de elaborar um conjunto de dados, construir um modelo estatístico baseado nesse conjunto de dados, e utilizar esse modelo para resolver de alguma forma, um problema prático.

3.2.2 Tipos de aprendizado

Antes de entrar a fundo nos tópicos de Aprendizado de Máquina, é importante explicar quais tipos de aprendizado existem e para que cada um serve, pois uma análise a respeito de cada um pode facilitar a escolha do método de aprendizado adequado para um determinado problema, no caso deste trabalho, o problema de classificar padrões em equipamentos de fins gerais através da captação de sinais via sensores.

Em relação aos tipos de aprendizado, existem dois tipos mais conhecidos, que serão melhor explicados nos próximos tópicos.

3.2.2.1 Aprendizado supervisionado

O aprendizado supervisionado consiste em utilizar um conjunto de dados composto de exemplos rotulados $\{(xi,yi)\}$ com i=1 até N, onde cada *xi* é um vetor que contém de 1 até M características e cada característica representa alguma informação relativa a *xi*. Como exemplo, pode-se imaginar que se cada *xi* representasse uma espécie de flor, suas características poderiam ser suas dimensões, cores, entre outros. E cada *yi* representa um conjunto finito de classes que *xi* pode pertencer, ou seja, seguindo o exemplo anterior, *yi* é o equivalente à espécie da flor.

A principal ideia do aprendizado supervisionado é utilizar um conjunto de dados nos moldes citados acima para treinar um modelo que ao receber como

entrada um vetor *xi* de características, pode deduzir a partir de um treino previamente realizado, as possíveis classes *yi* que *xi* se enquadra.

3.2.2.2 Aprendizado não supervisionado

Os algoritmos de aprendizado não supervisionado contém da mesma forma que o supervisionado, *conjuntos de dados* com diversas características, porém utilizamos esse tipo de aprendizado para extrair propriedades úteis dessas estruturas, como por exemplo, a distribuição probabilística que gerou o conjunto de dados, ou até mesmo encontrar semelhanças entre as diversas instâncias do mesmo (GOODFELLOW; BENGIO; COURVILLE, 2017).

3.2.3 Classificação versus regressão

Pode-se dividir os problemas do aprendizado supervisionado em problemas de Classificação ou Regressão, onde os problemas de Classificação tem como objetivo a partir de um modelo treinado, receber uma entrada x não rotulada e devolver essa mesma entrada com um rótulo/classe (Um exemplo de problemas de classificação são os algoritmos de detecção de *spam*).

Em problemas de classificação, o rótulo é membro de um conjunto finito de classes, sendo que se o tamanho do conjunto for dois, chamamos o problema de classificação binária, se houverem três ou mais classes chamamos esse problema de classificação multiclasse (BURKOV ,2019).

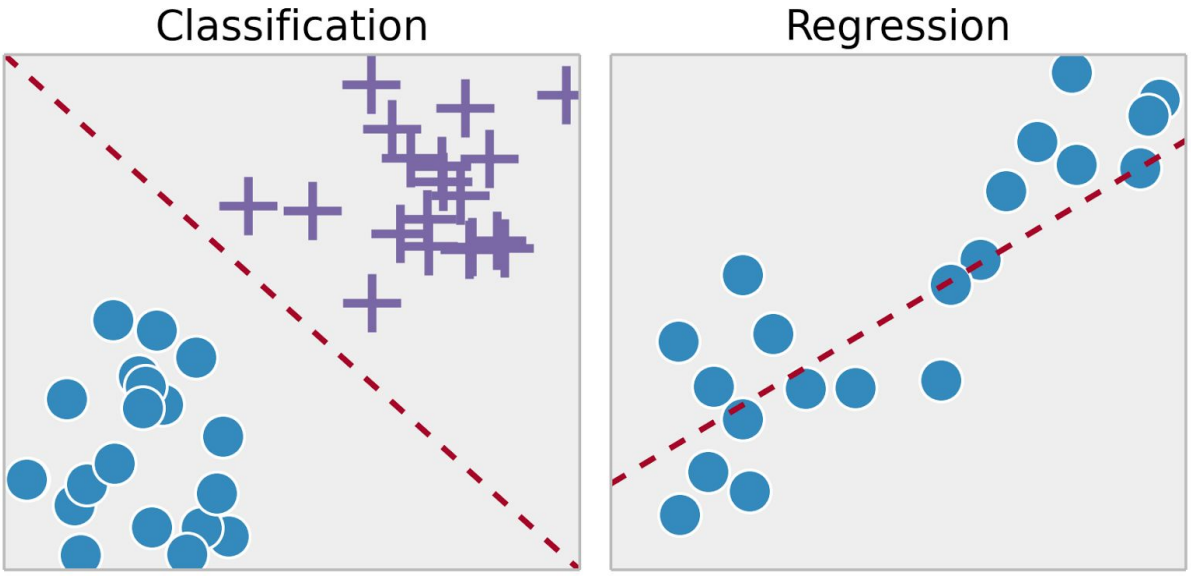
No caso da regressão, a mesma trata de resolver problemas normalmente relativos a predição ou a busca de uma relação linear entre valores(como por exemplo, verificar se existe uma relação entre idade, peso, dieta e expectativa de vida de uma pessoa).

Um exemplo de problema de regressão citado por Andriy Burkov (2019) é o problema de estimar o valor de um imóvel baseado nas suas características, como espaço físico, número de quartos, localização, entre outros. Sendo que neste tipo de

problema, não existe uma classe para o imóvel, mas sim um possível valor que está dentro de um intervalo contínuo.

A seguir está uma comparação da distribuição dos dados em problemas de classificação (à esquerda) e de regressão (à direita).

Figura 8. Representação de problemas de classificação e de regressão



Fonte: Eddel (2015)

O problema tratado foi intrinsecamente um problema de classificação, onde era preciso classificar os dados recebidos do ambiente em classes já conhecidas (microondas, torneira ligada, máquina de corte a laser).

3.2.4 Tratamento de dados

A seguir, serão abordados métodos de tratamento de dados relevantes para o presente trabalho, pois, com eles é possível padronizá-los, eliminar dados inválidos e realizar outros tipos de tratamento que possam torná-los mais consistentes, antes de realizar o treinamento do modelo.

3.2.4.1 Redução dos dados

A redução dos dados, trata-se da seleção dos atributos do conjunto de dados que possuem relevância para o processo de predição (GOLDSCHMIDT; PASSOS; BEZERRA, 2015). Um exemplo seria excluir atributos como id numérico ou nome, de uma base de pessoas em que se procura encontrar padrões baseado na idade.

3.2.4.2 Ausência e inconsistência de valores

Essa etapa do tratamento de dados é relevante para garantir a veracidade e coerência dos resultados, nela foram excluídas instâncias do conjunto de dados que possuem valores nulos que poderiam prejudicar a análise dos dados. Também seriam retiradas instâncias com inconsistências no preenchimento como valor errado (conhecido como um *outlier*) ou até mesmo tipo de dado errado (texto em campo numérico, por exemplo).

3.2.4.3 Normalização

É a abordagem escolhida pois após filtrar as duas opções com os padrões descritos na seção 3.2.4.4 foi o que se provou ser mais efetivo para a proposta deste trabalho.

A normalização de dados é o processo de definir os valores que uma característica do conjunto de dados possui em um intervalo (tipicamente [-1,1] ou [0,1]). Este processo utiliza a fórmula abaixo onde min(j) é o menor valor do intervalo, max(j) é o maior valor do intervalo e x(j) é o valor da característica atual.

$$\bar{x}^{(j)} = \frac{x^{(j)} - min^{(j)}}{max^{(j)} - min^{(j)}}$$

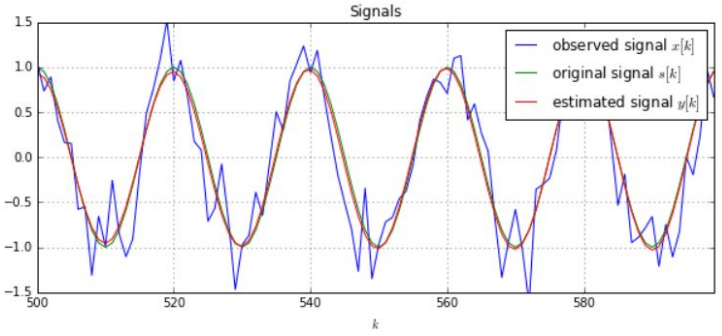
A normalização dos dados não é mandatória porém, Burkov (2019, p.46) afirma que essa prática pode resultar em um aumento na velocidade de aprendizagem do algoritmo.

3.2.4.4 Filtro de Wiener

O filtro de Wiener é um método cujo objetivo é a remoção de ruídos em um conjunto de dados, identificação de sistema, deconvolução e detecção de sinal. Através do seu uso é possível eliminar sinais de áudio ou remover ruídos em imagens. Para realizar essa filtragem é calculado o erro quadrático médio (útil na comparação de estimadores) que deve ser o menor possível no que diz respeito à função de transferência (relação entre entrada e saída de um sistema) gerando a menor variação entre o sinal original e o sinal estimado.

Conforme a figura à seguir, está demonstrado na cor azul o sinal observado, em verde o sinal original que é o valor real/ideal e em vermelho o sinal estimado através do filtro que segue bem próximo ao sinal original demonstrando sua eficácia.

Figura 9. Plotagem dados sinal observado, original e estimado



Fonte: Spors (2015)

3.2.5 Escolha do algoritmo de aprendizagem

Embora fosse interessante testar todos os algoritmos possíveis para encontrar o mais adequado para a resolução do problema, há uma série de filtros que pode-se fazer para filtrar as possibilidades. Burkov (2019, p.48,49) define esses filtros conforme abaixo:

- explicabilidade

  Caso o modelo precise ser explicado para um público não técnico, pode ser interessante utilizar algoritmos como o *KNN*, algoritmos de regressão linear ou árvore de decisão que embora não produzam os modelos mais precisos, a forma como fazem sua previsão é de menor complexidade comparado às outras estratégias;
- em-memória vs fora-da-memória

  Caso o modelo possa ser inteiramente carregado na memória *RAM* da máquina onde será rodado existe uma ampla variedade de algoritmos em conformidade, do contrário, é preferivel utilizar algoritmos que trabalhem de forma incremental, melhorando o modelo ao adicionar mais dados ao longo do tempo;
- número de instâncias e características

  Baseado no número de instâncias (uma instância é um agrupamento de atributos / características) presentes no conjunto de dados de treinamento e em quantas características / atributos (dados distintos pertencentes à instância como nome, altura, peso, salario e etc) cada instância possui, pode-se escolher entre estratégias que trabalhem bem com grande número de instâncias e de características como redes neurais e aumento de gradiente, ou estratégias que possuem certa limitação em sua capacidade e trabalhem melhor com um conjunto de dados menor e com menos características como o *SVM*;
- características numéricas ou de categorização

  É possível excluir algumas opções de algoritmo baseado na estrutura das características do conjunto de dados: se todas são numéricas, se todas são de categorização ou se são compostas por ambas. Isso se deve porque alguns algoritmos não podem manipular o conjunto de dados diretamente e seria preciso converter as características de categorização em números;
- linearidade dos dados

  Uma verificação a se fazer é se os dados são linearmente separáveis ou se os mesmos podem ser modelados utilizando um modelo linear, caso isso

aconteça, *SVM* com *kernel* linear, logística ou regressão linear costumam ser boas abordagens. Se não for possível, redes neurais profundas ou algoritmos de conjunto possivelmente terão um desempenho melhor;

- velocidade de treino

  Algumas estratégias possuem um longo tempo de treino como as redes neurais. Algoritmos mais simples como regressão linear, logística ou árvores de decisão levam menos tempo para o treino. Existem algumas bibliotecas que procuram otimizar as implementações de alguns algoritmos: algoritmos como de florestas aleatórias utilizam a disponibilidade de várias CPU para reduzir o tempo gasto; e

- velocidade de predição

  Por último resta saber em quanto tempo o modelo deve gerar as predições, sendo um modelo utilizado em produção que será utilizado várias vezes e deverá responder de forma rápida algoritmos como *SVM*, regressão linear, logística e alguns tipos de redes neurais podem ser escolhidos por gerarem previsões de forma muito rápida. Algoritmos como *KNN*, algoritmos de conjunto e redes neurais profundas ou recorrentes geram previsões em tempos maiores e seriam melhor utilizados em modelos cuja previsão pode levar mais tempo sem causar impacto no usuário do modelo. A melhor forma de avaliar o desempenho é testando os algoritmos no conjunto de validação.

3.2.6 Redes neurais e treinamento

As redes neurais possuem um destaque quando o assunto é um grande volume de dados, e devido à natureza do treinamento que foi realizado neste trabalho, o seu uso foi um diferencial em relação a outros algoritmos que não fazem uso desta técnica.

Nos tópicos abaixo serão explicados conceitos que ajudam a compreender como as redes neurais funcionam, desde o método do gradiente que auxilia na distribuição de pesos de neurônios, passando pelas funções de ativação que

definem se um neurônio deve ou não ser ativado, e finalizando com uma abordagem sobre a arquitetura *Multilayer Perceptron*.

3.2.6.1 Gradiente descendente

Para entender o método do gradiente (gradiente descendente) é necessário primeiramente entender o que é o gradiente. O gradiente de um vetor que indica sentido e direção e representa a intensidade da variação de uma função, ou seja, o quanto em relação a um ponto aquela função sofre alteração.

O método do gradiente tem como objetivo atingir de forma iterativa o mínimo local de uma função, esse valor representa o peso de um neurônio na classificação. Alguns neurônios podem ter importância maior que os outros no processo de classificação e isso pode ser observado pela forma que ele é ativado, com isso é possível minimizar o impacto desse tipo de neurônio na classificação (BISHOP, 2011; BURKOV, 2019).

3.2.6.2 Redes neurais

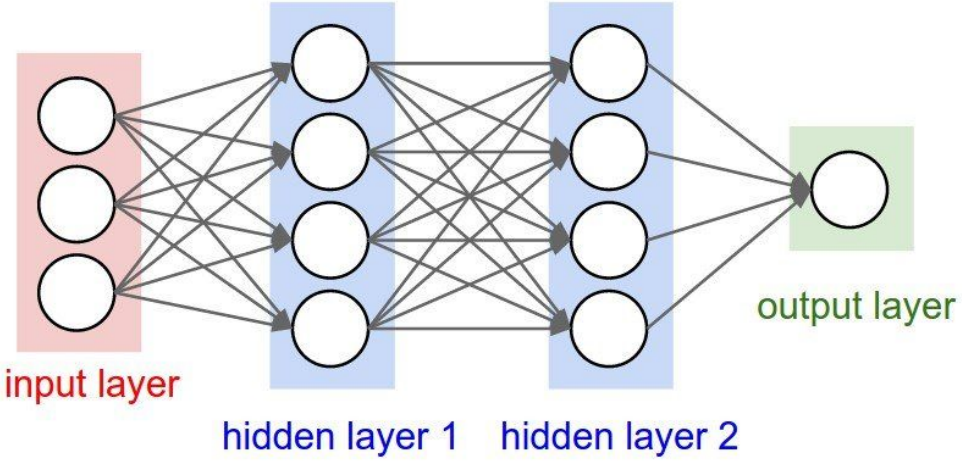
De acordo com Andriy Burkov (2019), uma rede neural pode ser definida como uma função matemática $y = f_{NN}(x)$, sendo essa, uma função aninhada, ou seja, composta por funções dentro de funções.

Quando se refere a funções aninhadas, deve-se ter em mente a noção de camadas em redes neurais, pois a quantidade de funções aninhadas está diretamente relacionada com a quantidade de camadas que uma rede neural possui. Logo uma rede neural de três camadas, por exemplo, pode ser expressada pela seguinte função $y = f_{NN}(x) = f_3(f_2(f_1(x)))$, onde f_1 e f_2 são funções vetoriais(funções matemáticas de uma ou mais variáveis cujo contradomínio é um conjunto de vetores multidimensionais), com a forma $f_l(z) = g_l(W_l z + b_l)$ onde l é o índice da camada, podendo variar de 1 até qualquer quantidade de camadas e g_l é uma função de ativação, fixada e normalmente não-linear, escolhida pelo analista de dados antes de

iniciar o treinamento. W_l e b_l tratam-se de parâmetros das camadas, obtidos utilizando o Método do Gradiente (explicado no tópico 3.2.6.1).

Conforme demonstrado na figura à seguir, uma rede neural possui layers de entrada, um número n de layers intermediários “escondidos” e o layer de saída com o resultado.

Figura 10. Exemplo de rede neural



Fonte: Mehra (2017)

3.2.6.3 Função de ativação

De acordo com a Data Science Academy (2019), a função de ativação corresponde a uma função matemática que tem como objetivo determinar se um neurônio deve ou não ser ativado, ou seja, ele determina qual a importância de cada informação para o neurônio por meio de uma transformação.

Não existe uma única função de ativação e cada uma pode ser aplicada para um determinado propósito.

As técnicas mais comuns são:

Função Binária: funciona por meio de *threshold*, onde é estipulado um valor mínimo para que neurônio seja ativado, qualquer valor inferior ao definido é zerado e ignorado por consequência. Esse método gera problemas de *backpropagation*, conforme explicado, o valor não pode ser zero.

Função Linear: f(x) = ax. Na função linear a variação da transformação é linear, isso gera um problema no *backpropagation*, por maior que seja o número de iteração, no final pode-se traduzir em uma única transformação linear. Essa método só é recomendado para tarefas simples onde o *backpropagation* não importa tanto.

Sigmóide: f(x) = 1/(1 + e^-x). A característica dessa função é empurrar os valores de Y para os extremos. O maior problema dessa função é que quando os gradientes estão muito pequenos e tende a zero, isso faz com que a rede pare de aprender. Essa função varia de 0 a 1.

Tanh: f(x) = 2/ (1/(1 + e^-x)) – 1. É uma versão da Sigmóide modificada, a diferença dessa função é que os valores podem variar de -1 a 1.

ReLu: f(x) = max(0, x). ReLU é linear. A característica mais marcante da ReLU é transformar todos os valores menor que zero em zero, assim deixando o neurônio desativado.

Softmax: sendo um tipo de camada de saída é uma versão da sigmóide modificada para trabalhar com saídas multidimensionais, em casos onde existem múltiplas classes.

3.2.6.4 Multilayer perceptron

O *MLP* (*Multilayer Perceptron)* é um algoritmo de redes neurais que foi utilizado no *Supersensor*, e por este motivo, o presente trabalho realizou os testes de algoritmos de classificação por essa arquitetura, sendo que os itens abordados nos subtópicos anteriores representam uma base teórica que auxilia no entendimento do funcionamento do *MLP*.

O objetivo do *MLP* é de aproximar alguma determinada função f(x), onde enquanto um classificador y = f(x) mapeia uma entrada x para uma saída y, o *MLP* define um mapeamento y = f(x; θ), que aprende o valor dos parâmetros representados por θ que resultam na melhor aproximação para a função y (GOODFELLOW; BENGIO; COURVILLE, 2017).

Devido ao fato de o *MLP* ser uma arquitetura de rede neural, os conceitos da seção 2.2.6.1 se aplicam à sua estrutura, ou seja, existe uma camada de entrada,

uma ou mais camadas intermediárias, chamadas de *hidden layers* e uma camada de saída. As funções que compõem a função de mapeamento do *MLP* podem utilizar as funções listadas na seção 3.2.6.3 para realizar o processamento de acordo com a natureza do problema a ser tratado, como por exemplo, o problema de classificação multiclasse (explicado na próxima seção), que pode ser resolvido utilizando a função *Softmax* na camada de saída.

3.2.7 Multi-label classification

O problema do *multi-label classification* surge ao tentar classificar algo que tem mais de um rótulo (classe) associado. Quando se trabalha com eventos do mundo real, existe a possibilidade de ter diversos rótulos para o mesmo conjunto de dados e não existe uma única classificação correta.

Uma solução para esse problema é associar o vetor de características a todos as classes identificadas, nesse momento ocorre um problema de classificação de múltiplas classes, sendo que após o treinamento é possível definir um *threshold, isso é*, se algum dos rótulos escolhidos for maior que o *threshold*, pode-se classificar com esses rótulos.

Outra solução para o problema, muito útil para os casos onde existem poucas classes possíveis, é gerar novas classes falsas que são as combinações das classes reais. Essa solução só é válida para os casos de poucas classes, caso contrário a quantidade de informação para treino teria de ser muito alta.

4 Desenvolvimento

Através do estudo da documentação do projeto Supersensor e dos quesitos levantados na Revisão Bibliográfica, foram identificadas as principais características para realização do trabalho.

Foi utilizado o microcontrolador Arduino, construído sob a arquitetura ARM, para orquestrar a captação através de sensores e enviar para uso posterior tanto no aprendizado quanto no processo de classificação a partir de técnicas de aprendizado

de máquina e classificação por redes neurais. Neste trabalho foram utilizados os sensores com maior peso na classificação geral do projeto Supersensor, conforme apontado na Figura 2, ou seja, os sensores de acelerômetro, microfone, e EMI além do sensor de luminosidade.

Os dados coletados são enviados via protocolo MQTT. A escolha deste protocolo foi feita baseada nos critérios da seção 3.1.3.2, principalmente pelo seu desacoplamento e assincronismo, o que possibilitou a classificação em tempo real de forma fluida e estável. Uma vez coletados, os dados são enviados para o Broker de onde a aplicação construída em Python 3 os coleta para persistir no banco de dados. Para o processo de treinamento do modelo, os dados coletados são pré processados de forma que o algoritmo possa ter maior acurácia (mais detalhes no item 4.3)

Foi criada uma interface de treinamento dos dados através de um chatbot feito utilizando a API do Telegram, para facilitar o rotulamento dos dados coletados de forma rápida e prática (detalhes no item 4.4).

Como visto no item 3.2.3, a solução proposta pelo presente trabalho pode ser classificado como um problema de classificação, onde é esperado que o algoritmo classifique os dados obtidos conforme classes já identificadas (microondas ligado, torneira aberta e etc). Não obstante, foi utilizado o treinamento supervisionado (visto no item 3.2.2) onde uma parte dos dados é classificada manualmente para que o algoritmo MLP (visto em 3.2.6.4) possa associar pesos as informações e classificar os dados conforme foi treinado.

Além de o algoritmo MLP ter sido utilizado no artigo que serviu de base para este trabalho, os critérios de escolha de algoritmo citados na seção 3.2.5 também foram relevantes. Era necessário um algoritmo que conseguisse lidar com uma grande quantidade de instâncias de treino em uma velocidade de predição rápida o suficiente para atender uma aplicação em tempo real, ou seja, conseguir realizar a classificação de um equipamento em um intervalo de poucos segundos. Dentro desse cenário, o algoritmo MLP se mostrou eficiente, motivando assim sua escolha.

A classificação em tempo real é enviada à um tópico no broker MQTT onde todos os clientes que estiverem assinado (*subscribed) o tópico* através do portal web receberão o resultado.

De forma parecida com o levantamento realizado na revisão bibliográfica, o desenvolvimento foi dividido em 3 partes principais e 2 auxiliares, sendo elas:

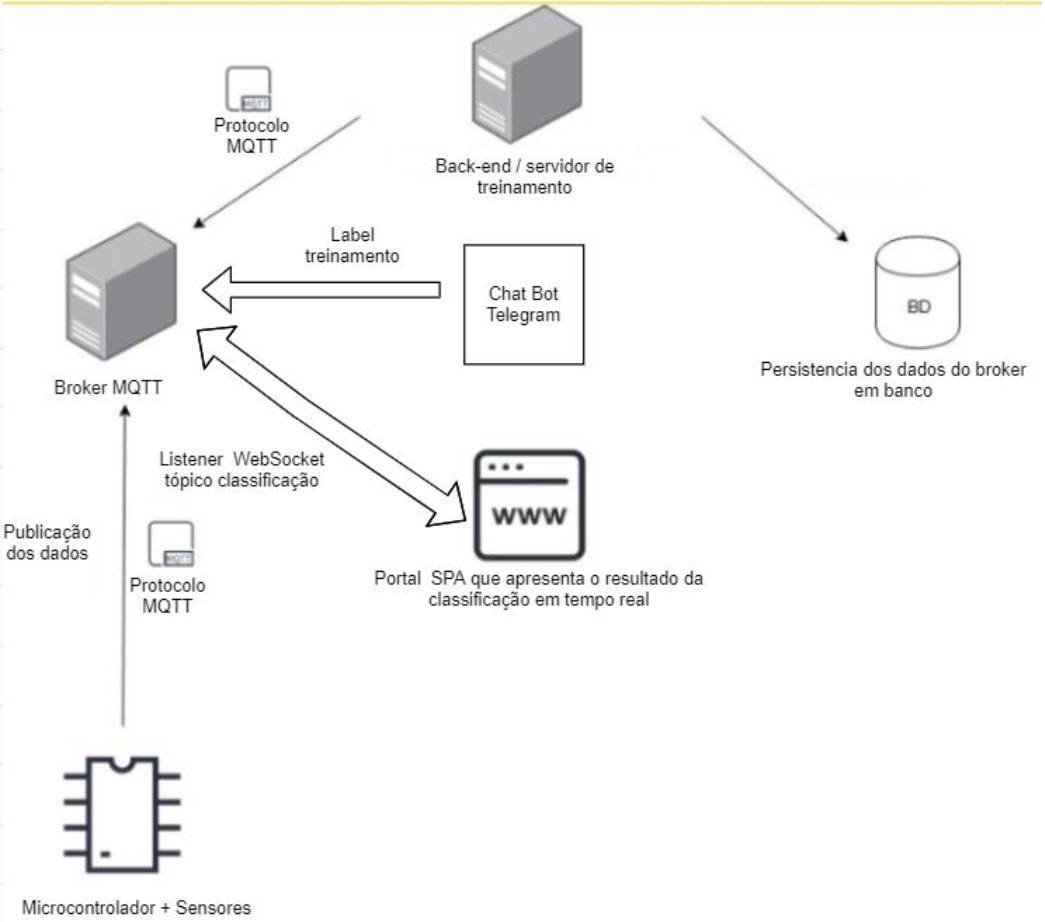
- Elaboração do circuito que responsável pela captação dos dados, validação destes dados, e envio para o servidor que responsável pela classificação;
- Configuração do microcontrolador para enviar dados utilizando o protocolo *MQTT*;
- Configuração do servidor que recebe os dados vindos dos sensores e realiza a classificação dos dados de acordo com o treinamento realizado anteriormente;
- Desenvolvimento da interface de visualização do fluxo de dados, controle de treinamento e exibição do resultado do algoritmo de classificação;
- Elaboração do circuito final;

A solução foi pensada de forma que é possível delegar a supervisão do treinamento à quem estiver alocado no recinto dos equipamentos utilizando um chatbot via Telegram com os comandos para treinamento.

O desenho da solução foi dividido entre a parte de IoT (placa, sensores e etc), a parte do Broker (MQTT), os serviços de classificação, treinamento e persistência, e por fim o chatbot (Telegram) de treinamento e a interface web.

Após o estudo dos trabalhos relacionados e do levantamento feito na revisão bibliográfica, a seguinte arquitetura foi desenvolvida, com uma seção de IoT com os sensores responsáveis pela captura dos dados, uma de Broker responsável pelo transporte das informações, uma seção de Aplicação cujo papel é coletar os dados do broker e persisti-los, uma seção de persistência dos dados e uma camada com o treinamento do modelo, a classificação dos resultados e a apresentação do resultado para o usuário via portal.

Figura 11 - Arquitetura utilizada na solução



Fonte: Autoria própria

4.1 Elaboração do circuito e validação dos dados dos sensores (IOT)

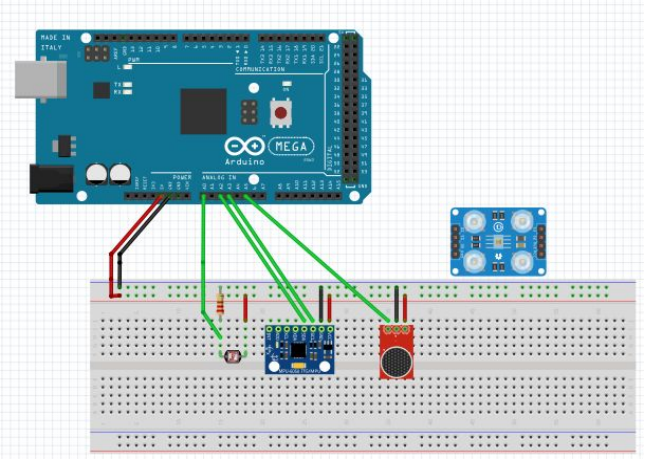
Nessa etapa o circuito foi elaborado utilizando:

- microcontrolador arduino;
- acelerômetro;
- microfone/sensor de som;
- sensor *EMI;*
- sensor de luz;

De forma que foi possível ter uma noção melhor do produto final, e realizar os testes necessários para verificar como os sensores funcionam juntos e, a partir dos

resultados dos testes, verificar a necessidade de alterar o circuito para melhor funcionamento dos mesmos e realizar os tratamentos.

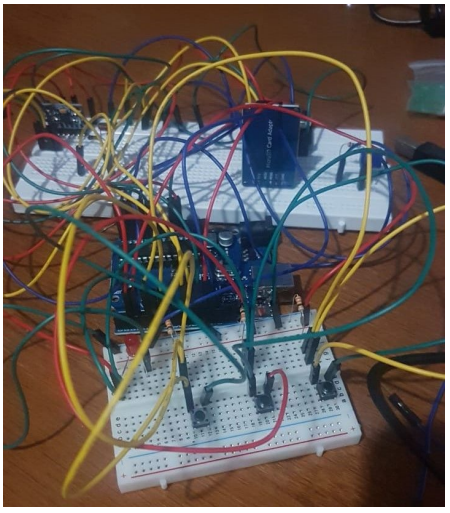
Figura 12 - Circuito confeccionado preliminarmente



Fonte: Autoria própria

O circuito foi conectado ao arduino e os sensores conectados na placa. Sendo que todos os testes realizados anteriormente foram levados em conta para escolher a posição de cada sensor.

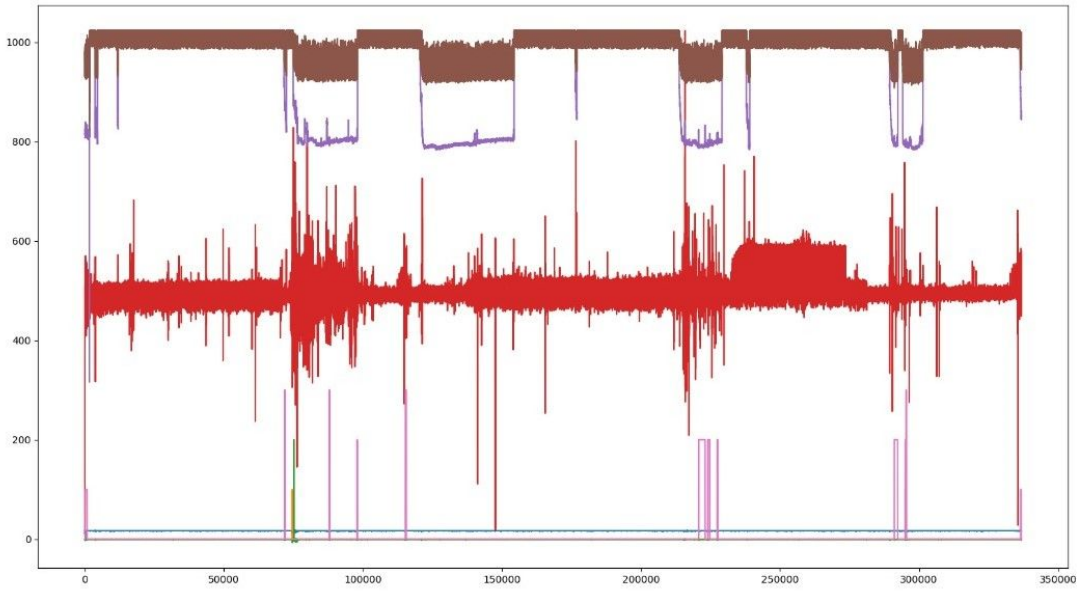
Figura 13 - Placa de circuitos física



Fonte: Autoria própria

Foi possível acompanhar os dados recebidos por cada sensor de forma visual utilizando a ferramenta de plotting do Python após a coleta conforme a Figura 13.

Figura 14 - Acompanhamento dos dados



Fonte: Autoria própria

4.2 Configuração do broker MQTT

A configuração do broker *MQTT* envolveu os seguintes passos:

- **Configuração do Arduino:** essa etapa envolveu configurar uma conexão sem-fio no Arduino, e elaborar o código responsável por canalizar os dados lidos pelos sensores para os tópicos *MQTT* e enviá-los para o *broker*;
- **Configuração do broker no servidor:** nesta etapa é realizada a configuração do broker no servidor que também foi o servidor responsável pelo treinamento e classificação dos dados recebidos; e
- **Teste do canal de comunicação:** com o Arduino e o *broker* devidamente configurados, foi utilizado um cliente *MQTT* para verificar o fluxo de dados, como também a latência e outros aspectos da conexão afim de realizar ajustes para melhorar a qualidade da comunicação de acordo com a necessidade do projeto.

Foram definidos dois tópicos no broker, o primeiro recebe os dados vindos dos sensores (já com um tratamento realizado no Raspberry pi) para

que a Rede Neural seja treinada, e o segundo que recebe o resultado da classificação em tempo real.

4.3 Recepção dos dados do lado do servidor e treinamento (Middleware)

Ao iniciar os testes de modelos para classificação dos dados de entrada coletados, foi necessário fazer uma análise exploratória de possíveis modelos de rede neural MLP, ou seja, uma configuração para as camadas intermediárias (quantidade de camadas, quantidade de neurônios e funções de ativação) e para a camada de saída, que poderia retornar uma classificação com a classe com maior probabilidade de acerto, ou um conjunto das classes com maiores probabilidades. No primeiro momento, optou-se pela primeira alternativa, onde a rede retornaria a classe com maior probabilidade dada uma certa entrada.

Para a construção dos primeiros modelos, foi utilizado um script python para testar várias combinações de quantidades de camadas ocultas e quantidade de neurônios. Com essa estratégia foi possível encontrar alguns modelos que alcançaram uma taxa de acerto que variou entre 40% e 50%, o que apesar de ser um progresso significativo, ainda poderia ser melhorado.

Após a primeira leva de testes, realizamos uma segunda coleta de dados e, assim que esses foram rotulados, foi realizado o tratamento dos dados (citado na seção 3.2.4) buscando remover atributos desnecessários e elementos com ausência de valores (assim, instâncias que prejudicariam o treinamento foram removidas).

Os dados são armazenados em um arquivo csv que é carregado no script que transforma o texto em um vetor de parâmetros numéricos para que possam ser separados em um array para cada sensor. Em seguida, é calculado o desvio padrão das 100 últimas entradas e é aplicado o *filtro de Wiener* visto na seção 3.2.4.4 (um dos filtros de ruídos disponíveis na biblioteca python *scipy*) para retirar possíveis ruídos dos dados, deslocamos 1 elemento para a direita e repetimos o processo para os próximos dados.

Os dados tratados são agrupados no formato de matriz:

([[sensor1, sensor2, …, sensorN],
[sensor1, sensor2, …, sensorN]])

Então são separadas o mesmo número de ocorrências de todas as classes presentes na base para que a rede não fique com tendência a classificar uma classe específica conforme a figura 15 onde um excesso de ocorrências onde não havia nada ligado induziu a classificação como nenhum equipamento ligado em 100% dos casos.

Figura 15. Base de dados sem tratamento de número de ocorrências

		Classificado			
		Nenhum	Microondas	Torneira	Filtro de água
Classe real	Nenhum	100,00%	0,00%	0,00%	0,00%
	Microondas	100,00%	0,00%	0,00%	0,00%
	Torneira	100,00%	0,00%	0,00%	0,00%
	Filtro de água	100,00%	0,00%	0,00%	0,00%

Fonte: Autoria própria (2019)

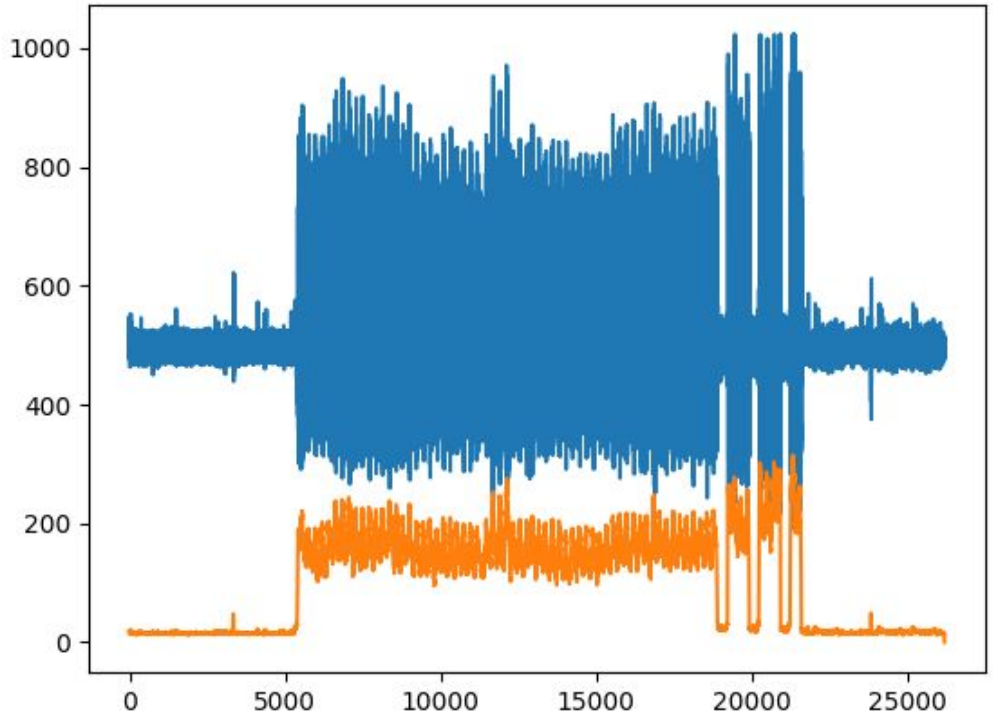
O próximo passo foi realizar uma análise exploratória a partir do algoritmo *Multilayer Perceptron (MLP)* utilizando o scikit do Python, para definir os valores utilizados como parâmetros de entrada do algoritmo observando como suas variações afetam a acurácia (número de layers e etc).

Os dados foram separados em dois grupos, grupo de treino com 70% dos dados, e grupo de teste com 30% dos dados. Os dados foram então normalizados via scikit e chegamos a seguinte configuração através da análise exploratória:

```
MLPClassifier(solver='lbfgs', alpha=1e-5, hidden_layer_sizes=(7,4), random_state=1)
```

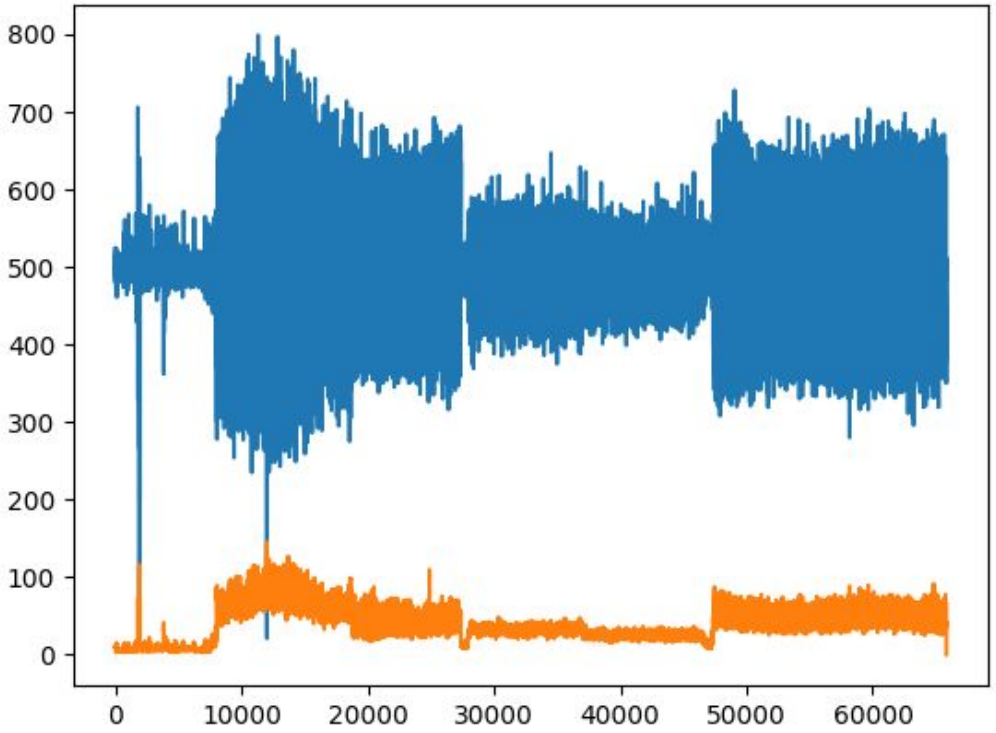
Assim foi possível avaliar como o algoritmo se comporta, e confirmar a aderência do algoritmo ao problema de classificação proposto neste trabalho acompanhando os acertos na base de teste.

Figura 16. Tratamento do som em ambiente doméstico, azul: sem tratamento, laranja: tratado



Fonte: Autoria própria (2019)

Figura 17. Tratamento do som da CNC, azul: sem tratamento, laranja: tratado

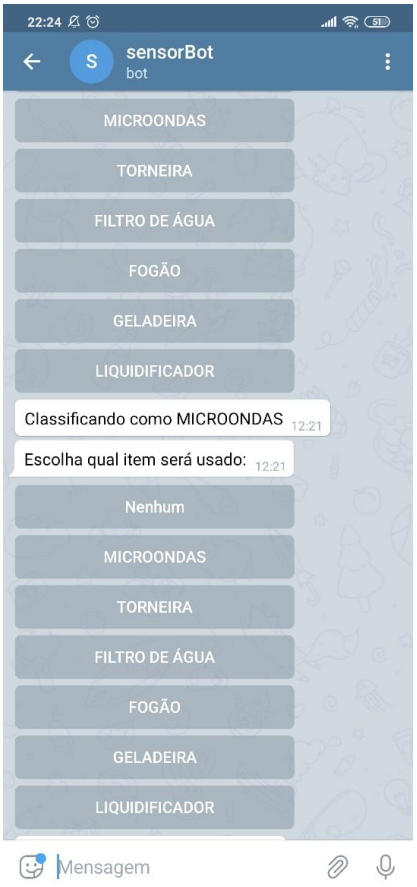


Fonte: Autoria própria (2019)

4.4 Interfaces treinamento e classificação

Como interface para o treinamento do modelo, foi confeccionado um Chatbot (sensorBot) com a *API (Application Interface)* do Telegram, que permite marcar os equipamentos ligados para que qualquer um consiga realizar o treinamento. A partir daí, o serviço de treinamento no *back-end* passa a marcar a classe nos dados recebidos no *broker* até que seja dado como desligado.

Figura 18 - Chatbot Telegram para treinamento



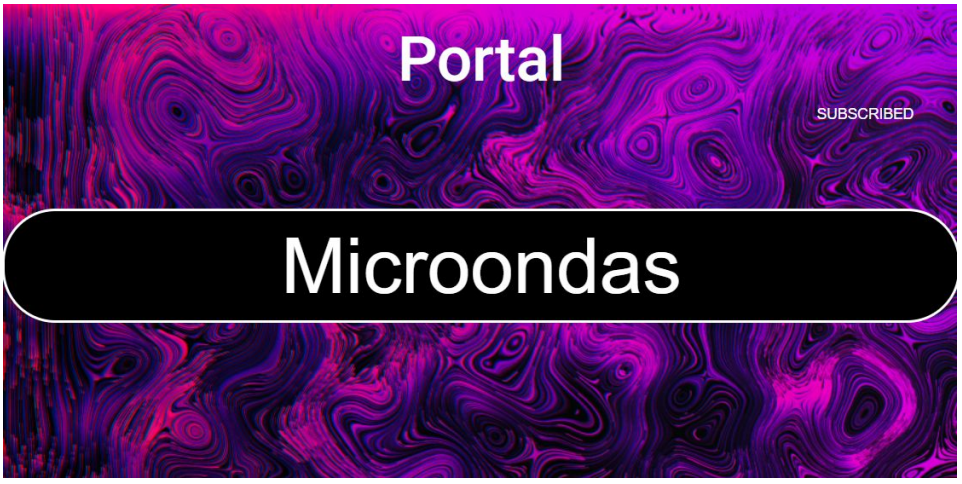
Fonte: Autoria própria (2019)

Durante a classificação aplicamos os mesmos tratamentos do tópico 4.3 com algumas diferenças em relação à quantidade de dados analisados. Recebemos 50 instâncias do sensor ao invés de 100, aplicamos o desvio padrão e o filtro a cada 25

instâncias. As 50 entradas tratadas são classificadas com o modelo treinado e a label de maior ocorrência é considerada a classificação.

Para receber o resultado da classificação em tempo real, foi criada uma página web *SPA (Single Page Application)* conectada diretamente no tópico do broker com a classificação para diminuir o delay na apresentação do resultado. Como a página é *SPA*, o componente faz o *data binding* do conteúdo e atualiza os dados de todos que estão conectados automaticamente, sem precisar atualizar a página. O empacotamento do front foi feito com as opções de *tree shaking* (limpeza do código que deixa apenas o código útil), *minify* (redução do código) e *uglify* (para ofuscar os dados do portal).

Figura 19 - Portal Web no computador



Fonte: Autoria própria (2019)

4.5 Cenários de Teste

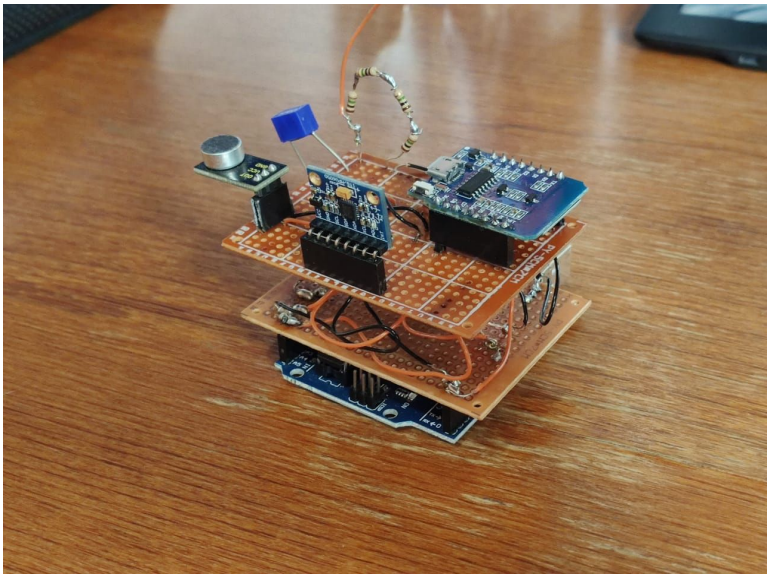
Os testes foram realizados em dois ambientes para avaliar a versatilidade da solução em cenários diversos conforme nosso objetivo e para fim de comparação com o Supersensor.

Os testes iniciais foram feitos em uma cozinha pequena durante sua utilização cotidiana, um protótipo da placa foi colocada sobre uma bancada, onde os equipamentos em uso foram classificados por meio da interface desenvolvida e descrita no tópico 4.4 de modo que não influenciasse os dados coletados. O primeiro passo foi coletar uma base de dados grande com todos equipamentos possíveis

para determinar quais equipamentos tinham efeito sobre nossa placa. Os equipamentos foram classificados inicialmente em 4 classes (Nenhum, Microondas, Torneira, Filtro de água) em um total de 1.481.157 instâncias.

Para garantir que a posição dos sensores não estava influenciando a classificação entre os dias, foi construído um protótipo com os sensores soldados a uma placa (Figura 20) e coletados diversas bases de dados pequenas ao longo dos dias para fins de confirmação, sendo possível observar a almejada estabilidade nos resultados obtidos.

Figura 20 - Circuito finalizado

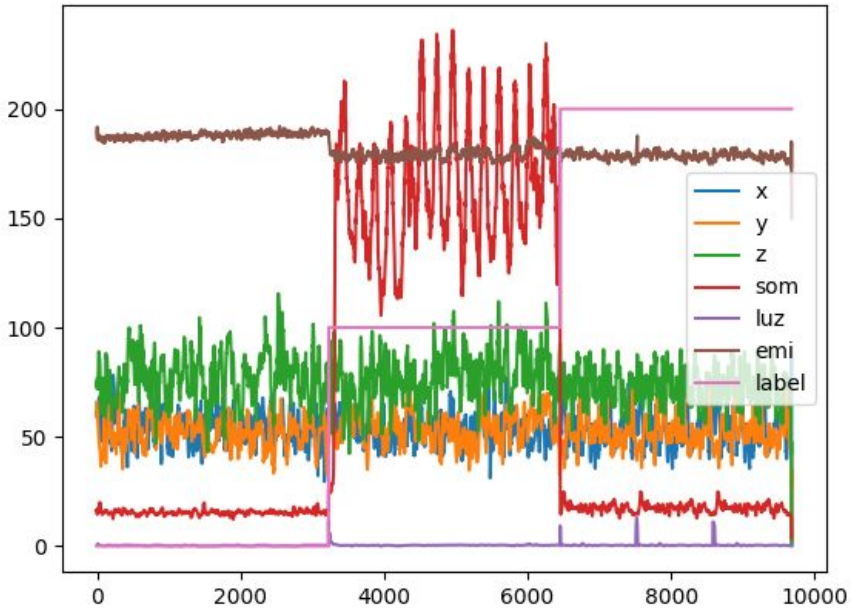


Fonte: Autoria própria (2019)

Por fim, pré-processando os dados conforme descrito na seção 4.3 alcançou-se um protótipo que se mostrou robusto quanto aos dados coletados, saindo de uma precisão de cerca de 50% quando os dados estavam brutos para uma média de 84% considerando apenas as classes que geram sinais no ambiente que podem ser captados pelos sensores.

Na figura 21 estão presentes os dados recebidos pelos sensores durante a execução dos testes no ambiente da cozinha e a classe identificada como label no gráfico sendo label 0 igual a nenhum equipamento ligado, label 100 igual ao microondas ligado e label 200 igual à torneira aberta.

Figura 21. Plotagem dos dados oriundos da cozinha



Fonte: Autoria própria (2019)

O segundo cenário foi a oficina do Centro Universitário Senac - Santo Amaro com o objetivo de classificar as duas máquinas CNCs de corte a laser em funcionamento. A placa construída foi posicionada embaixo de uma das máquinas e ambas as máquinas foram ligada por alguns minutos individualmente e depois juntas para gerar um base de dados de teste aplicando os mesmos tratamentos já descritos atingindo os seguintes resultados presentes na Figura 22. Os dados coletados vistos na Figura 23 demonstraram ativação de forma diferente nos sensores em relação ao cenário anterior identificado na Figura 21 mantendo uma boa precisão na classificação, superior à 80% para as máquinas ligadas individualmente e uma precisão de 58% para as duas máquinas juntas.

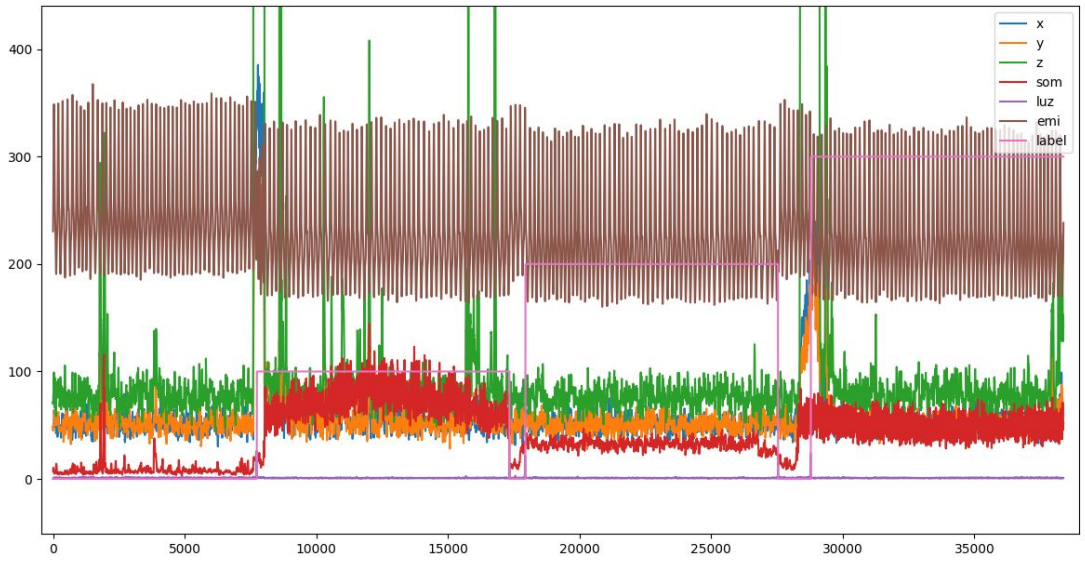
Figura 22. Matriz de confusão do teste com as CNCs

Classe real	Classificado: Nenhum	Máquina 1	Máquina 2	Máquina 1 e 2
Nenhum	93,87%	1,50%	3,20%	1,43%
Máquina 1	0,97%	80,71%	1,28%	17,04%
Máquina 2	1,42%	0,00%	86,84%	11,74%
Máquina 1 e 2	2,08%	17,40%	22,06%	58,47%

Fonte: Autoria própria (2019)

Na figura 23 estão presentes os dados recebidos pelos sensores durante a execução do teste na oficina e a classe identificada como label no gráfico sendo label 0 igual à nenhum equipamento ligado, label 100 igual à máquina CNC 1 ligada, label 200 igual à máquina CNC 2 ligada e por fim label 300 igual às máquinas 1 e 2 ligadas ao mesmo tempo..

Figura 23. Plotagem dos dados oriundos da oficina



Fonte: Autoria própria (2019)

5. Conclusão

Este projeto foi motivado a partir da análise do artigo do Supersensor, onde os autores demonstram a capacidade de monitorar equipamentos não inteligentes em diversos cenários sem a necessidade de sensoriar cada um deles individualmente, atingindo uma precisão elevada de forma econômica e mantendo a privacidade dos dados não só pelo tratamento aplicado mas também em relação a ausência de sensores de diversas marcas.

Nossos objetivos eram realizar uma prova de conceito de uma placa capaz de monitorar ambientes distintos sem modificar o dispositivo e comparar o desempenho e diferenças observados com os resultados do Supersensor.

Ao longo do trabalho, foram usados os conceitos estudados no levantamento bibliográfico para construir uma solução que cumprisse com os objetivos do trabalho. Considerando que os resultados obtidos nos dois cenários atingiram uma precisão aceitável conforme Figuras 22 e 25, foi possível demonstrar a versatilidade da solução, semelhante ao que foi observado no artigo do Supersensor.

Quando aplicado o resultado do trabalho nos cenários descritos no tópico 4.5, foi possível observar que a combinação de sensores gera uma maleabilidade nas classificações possíveis em diversos ambientes, na qual cada sensor tem seu papel dependendo diretamente do propósito para qual está sendo treinado e por consequência, da qualidade dos dados captados por esses sensores. Na Figura 24 é possível observar que o Filtro de água não gerava características suficientes no ambiente para ser detectado pelos sensores e atrapalhava a classificação como um todo, sendo assim excluído nas outras classificações, o que melhorou o resultado total da rede conforme Figura 25.

Figura 24. Matriz de confusão da cozinha com pouca precisão

Classe real	Classificado: Nenhum	Microondas	Torneira	Filtro de água
Nenhum	39,93%	4,56%	15,67%	39,84%
Microondas	5,04%	78,87%	14,36%	1,73%
Torneira	17,25%	38,66%	27,32%	16,76%
Filtro de água	8,33%	16,74%	10,29%	64,63%

Fonte: Autoria própria (2019)

Figura 25. Matriz de confusão da cozinha sem filtro de água

Classe real	Classificado: Nenhum	Microondas	Torneira
Nenhum	91,12%	0,00%	8,88%
Microondas	0,00%	100,00%	0,00%
Torneira	16,00%	0,00%	84,00%

Fonte: Autoria própria (2019)

Outro fator importante para uma classificação eficiente é um tratamento de dados efetivo, a base de dados coletada sem um balanceamento da quantidade de classes de cada tipo para treino criou um tendência para uma única classe conforme descrito no tópico 4.3, além da necessidade dos outros tratamentos para otimizar características dos dados disponíveis que podem estar perdidas em ruído.

Sendo assim, é possível concluir que com uma variedade de sensores de boa qualidade é possível construir um rede neural capaz de classificar com uma precisão aceitável os eventos em um ambiente conforme foi demonstrado também pelo Supersensor.

Para projetos futuros, ficam em aberto testes com uma quantidade maior de sensores e tratamentos mais específicos para cada tipo de sinal recebido pelo microcontrolador.

6 Referências

Livros

BISHOP, Christopher M.. Pattern Recognition and Machine Learning, Springer, 2011.

BURKOV, Andriy. The Hundred-Page Machine Learning Book. Burkov, 2019.

CARR, Joseph. The Technician's EMI Handbook: Clues and Solution. Newnes, 2000.

CIRANI, Simone; FERRARI, Gianluigi; PICONE, Marco; VELTRI, Luca. Internet of Things: Architectures, Protocols and Standards. Wiley, 2019.

EARGLE, John. The Microphone Book, Second Edition: From mono to stereo to surround - a guide to microphone design and application: Focal Press, 2004.

GERTZ, Emily; JUSTO, Patrick Di. Environmental Monitoring with Arduino: Building Simple Devices to Collect Data About the World Around Us. Maker Media, 2012.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron; Deep Learning. Goodfellow; Bengio; Courville, 2017.

GOLDSCHMIDT, R.; PASSOS, E.; BEZERRA, E. Data Mining: Conceitos, técnicas, algoritmos, orientações e aplicações. 2. ed. [S.l.]: Elsevier Editora, 2015.

GUPTA; REYNOLDS; PATEL. ElectriSense: Single-Point Sensing Using EMI for Electrical Event Detection and Classification in the Home, 2010.

HILLAR, Gastón. MQTT Essentials - A Lightweight IoT Protocol. Packt Publishing Ltd, 2017.

KARAGIANNIS, Vasileios; CHATZIMISIOS, Periklis; VAZQUES-GALLEGO, Francisco; ALONSO-ZARATE, Jesus. A Survey on Application Layer Protocols for the Internet of Things. Transaction on IoT and Cloud Computing, 2015.

STALLINGS. William. Arquitetura e Organização de Computadores. 8. ed. [S.l.]: Pearson, 2010.

Websites

Data Science Academy. Deep Learning Book, 2019. Capítulo 8 – Função de Ativação. Deep Learning Book. Disponivel em: <http://deeplearningbook.com.br/funcao-de-ativacao/>. [acessado em 12/05/2019]

DAVID, CHIMA, UGOCHUKWU, OBINNA. Design of a Home Automation System Using Arduino <https://pdfs.semanticscholar.org/3df1/ebbe4ad3c91c54810540e17670805676041e.pdf>. [acessado em 19/05/2019].

Documentação para scipy.signal.wiener disponivel em : https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.wiener.html#scipy.signal.wiener [acessado em 01/12/2019]

EDDEL. IS MACHINE LEARNING THE NEW EPM BLACK? Representação de problemas de classificação e de regressão <https://scorecardstreet.wordpress.com/2015/12/09/is-machine-learning-the-new-epm-black/>. [acessado em 07/05/2019].

HETU, Robert. By 2020, 50% of everyday essential products will be autoreplenished through IoT. 2016. <https://blogs.gartner.com/robert-hetu/gartner-predicts-2020-50-everyday-essential-products-will-autoreplenished/>. [acessado em 19/05/2019].

HILL, Kashmir; MATTU, Surya. Palestra proferida no TED Talks, TED2018. Disponível em: https://www.ted.com/talks/kashmir_hill_and_surya_mattu_what_your_smart_devices_know_and_share_about_you/up-next#t-404383 [acessado em 24/04/2019]

LAPUT; ZHANG; HARRISON. Synthetic Sensors: Towards General-Purpose Sensing Tabela de sensores presentes no Supersensor <http://www.gierad.com/assets/supersensor/supersensor.pdf>. [acessado em 08/05/2019].

_____ Synthetic Sensors: Towards General-Purpose Sensing Interface do *Supersensor* classificando um microondas em funcionamento <http://www.gierad.com/assets/supersensor/highres/28_microwave_running.jpg>. [acessado em 19/05/2019].

LDR Datasheet Modelo: 3190. Disponivel em: https://www.sunrom.com/get/443700 [acessado em: 19/05/2019]

LEE, In; LEE, Kyoochun. The Internet of Things (IoT): Applications, investments, and challenges for enterprises - Business Horizons Volume 58, Issue 4, July–August 2015, Pages 431-440 disponivel em: https://www.sciencedirect.com/science/article/pii/S0007681315000373 [acessado em 19/05/2019]

MEHRA. Neural Network Architecture Reference
Exemplo de rede neural <https://www.researchgate.net/figure/Neural-Network-architecture-Reference-http-cs231ngithubio-neural-networks-1_fig11_325359390>. [acessado em 21/05/2019].

MPU-6000/MPU-6060 EV Board User Guide, Disponivel em: <https://www.invensense.com/wp-content/uploads/2015/02/MPU-6000-EV-Board1.pdf>. [acessado em 15/05/2019]

SANDERSON, Grant. Video aula: Descida de gradiente, como as redes neurais aprendem | Capítulo 2, aprendizado profundo.
Disponível em: https://youtu.be/IHZwWFHWa-w [acessado em 12/05/2019

_____ Video aula: What is backpropagation really doing? | Deep learning, chapter 3.
Disponível em: https://youtu.be/Ilg3gGewQ5U [acessado em 12/05/2019]

SILVA, Alisson; PEREIRA, Fábio. ARQUITETURA PARA CONSUMO E ANÁLISE DE DADOS EM TEMPO REAL APLICADA A REDES DE SENSORES IOT
Disponível em: http://www.sbrc2018.ufscar.br/wp-content/uploads/2018/04/04-182343.pdf [acessado em 02/06/2019]

SPORS, Sascha. The Wiener Filter
Disponível em: https://dsp-nbsphinx.readthedocs.io/en/nbsphinx-experiment/random_signals_LTI_systems/wiener_filter.html [acessado em 02/12/2019]